

Redes neuronales para bioinformática

TC-CH10 – Técnicas Computacionales en Biología

Edición 2 · 2026-09-03

Pregunta del tema. ¿Cómo puede un programa aprender por sí mismo a distinguir un enhancer activo de uno inactivo a partir de señales epigenéticas, y qué garantías tenemos de que lo que ha aprendido sirva en un hospital distinto del que produjo los datos?

Producto final. El cálculo a mano, verificado con un oráculo, de una pasada completa de entrenamiento de una red pequeña, incluido el paso hacia atrás hasta la capa oculta, más una auditoría del diseño de validación que evite la fuga de datos por paciente.

Mapa del tema

1. De modelos explícitos a representaciones aprendidas	1
2. El Perceptrón y la no linealidad	2
3. Entrenamiento, pérdida y retropropagación	3
4. Caso de estudio resuelto: predicción de enhancers	4
5. Sobreajuste y alta dimensionalidad en ciencias ómicas	7
6. Responsabilidad, ética y límites del aprendizaje automático	8
7. La operación peligrosa: fugas de datos y saturación	9
8. Actividad de depuración: errores en la retropropagación	9
9. Síntesis operativa y tablas de diagnóstico	10
10. Glosario conceptual	10
11. Cierre del libro: la caja de herramientas computacionales	10
12. Fuentes y reproducibilidad	11

Cómo usar este tema

Este capítulo cierra el libro y hace algo que ninguno de los anteriores necesitaba: además de explicar cómo funciona el método, dedica una sección entera a cuándo no hay que fiarse de él. Las dos partes son igual de importantes, y la segunda es la que distingue a quien sabe usar una red de quien sabe invocarla.

Al terminar sabrá: seguir a mano una pasada completa de entrenamiento de una red pequeña, hacia delante y hacia atrás; explicar por qué una red sin funciones de activación no lineales es equivalente a una regresión lineal; reconocer el régimen en que hay más variables que muestras y qué le hace al modelo; y auditar un diseño de validación buscando por dónde se filtra la información del conjunto de prueba.

La ruta **esencial** son el mapa del aprendizaje automático, la neurona y sus activaciones, la pérdida, el descenso de gradiente, el caso resuelto y el marco de responsabilidad. La ruta de **estudio** añade el sobreajuste con más variables que muestras, la saturación, la actividad de depuración y el anexo.

1 De modelos explícitos a representaciones aprendidas

ESENCIAL A lo largo de los capítulos anteriores hemos construido modelos con reglas, matrices o distribuciones explícitamente diseñadas por el analista:

- En **Naïve-Bayes** (TC-CH08), asumimos independencia condicional y estimamos frecuencias directas de nucleótidos.
- En **Cadenas y Modelos Ocultos de Markov** (TC-CH08), fijamos matrices de transición estocásticas para capturar dependencias de orden 1.
- En **Alineamiento de secuencias** (TC-CH09), definimos a mano matrices de sustitución (PAM, BLOSUM) y penalizaciones afines de hueco.

Sin embargo, cuando nos enfrentamos a problemas biomédicos complejos —como integrar simultáneamente múltiples pistas epigenéticas, perfiles de expresión génica masiva o predecir la actividad de regiones reguladoras

distales (*enhancers*)—, las interacciones biológicas no lineales y combinatorias resultan difíciles de codificar a mano con reglas fijas.

Este capítulo cierra el libro introduciendo las **redes neuronales Artificiales**: arquitecturas parametrizadas capaces de **aprender representaciones jerárquicas y fronteras de decisión complejas directamente a partir de los datos**.

Las redes neuronales representan la transición desde modelos probabilísticos artesanales y reglas de puntuación fijas hacia arquitecturas parametrizadas que aprenden representaciones jerárquicas y fronteras de decisión no lineales directamente a partir de datos biológicos.

2 El Perceptrón y la no linealidad

2.1 Dónde estamos: un mapa del aprendizaje automático

ESENCIAL La **inteligencia artificial** es el campo. Dentro de ella, el **aprendizaje automático** construye modelos a partir de datos en vez de reglas escritas a mano. Dentro de este, el **aprendizaje profundo** usa redes neuronales con muchas capas. Y la **inteligencia artificial generativa** es la familia de modelos profundos que producen datos nuevos, texto, imágenes o secuencias, en lugar de clasificarlos.

Un modelo aprende en dos fases: **entrenamiento**, donde ve ejemplos y ajusta sus parámetros, e **inferencia**, donde predice sobre casos nuevos. Si los ejemplos llevan etiqueta, esta región es un enhancer y esta no, el aprendizaje es **supervisado**; si no la llevan y el modelo busca estructura por sí mismo, agrupando pacientes con perfiles parecidos, es **no supervisado**.

Hay otra distinción que conviene tener clara. Un modelo **discriminativo** aprende la frontera entre clases: dado este perfil, ¿enfermo o sano? Un modelo **generativo** aprende cómo son los datos de cada clase y puede fabricar ejemplos nuevos. El clasificador de Bayes del capítulo de probabilidad es generativo; las redes de este capítulo son discriminativas. Los grandes modelos de lenguaje son generativos: predicen la siguiente palabra calculando probabilidades.

Tarea	Modelos habituales	Cuándo
Clasificar pacientes a partir de expresión génica	máquinas de vectores soporte, bosques aleatorios, redes	supervisado, hay etiquetas
Reducir dimensiones y visualizar cohortes	análisis de componentes principales, t-SNE, UMAP	no supervisado, exploración
Predecir actividad reguladora desde secuencia y epigenética	redes neuronales	supervisado, interacciones no lineales

Concepto. Las redes neuronales no sustituyen a las otras familias: son una opción más. Conviene cuando hay muchos datos y las interacciones entre variables son complejas, y son mala elección cuando hay pocos ejemplos, que es justo el régimen habitual en biomedicina y del que se ocupa la sección de sobreajuste.

2.2 De la neurona biológica a la unidad de cómputo

El **perceptrón** [2] es la unidad básica del aprendizaje profundo, inspirado en el funcionamiento electroquímico de las neuronas biológicas:

1. **Dendritas (Entradas)**: Reciben el vector de características biológicas $\mathbf{x} = [x_1, x_2, \dots, x_d]^T$ (p. ej., niveles de metilación o accesibilidad cromatínica).
2. **Soma (Combinación lineal)**: Realiza una suma ponderada de las entradas modulada por los pesos sinápticos $\mathbf{w}$ y un sesgo basal (*bias*) b :

$$z = \sum_{i=1}^d w_i x_i + b = \mathbf{w}^T \mathbf{x} + b$$

3. **Axón (Activación y Salida)**: Transforma el potencial lineal z mediante una **función de activación no lineal** $f(z)$ para emitir la activación final $a = f(z)$.

El perceptrón artificial modela una neurona biológica calculando una combinación afín $z = \sum w_i x_i + b$ y aplicando una función de activación $a = f(z)$ para generar la salida.

2.3 La necesidad matemática de la no linealidad

Si utilizáramos funciones de activación lineales ($f(z) = c \cdot z$), una red neuronal con múltiples capas ocultas colapsaría matemáticamente a una única transformación afín equivalente:

$$y = W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2) = W_{eq}x + b_{eq}$$

Una red puramente lineal no puede resolver problemas que no sean linealmente separables, y el caso canónico es la función XOR. Su equivalente biológico es un gen que se activa cuando está presente uno de dos factores, pero *no* cuando coinciden los dos, porque compiten por el mismo sitio de unión. Dibuje los cuatro casos en un plano y compruébelo: no hay ninguna recta que separe los dos que activan de los dos que no.

Las funciones de activación no lineales (Sigmoide, ReLU, Tanh) son matemáticamente indispensables porque una composición de capas puramente lineales colapsa a una única transformación lineal, incapaz de resolver problemas no linealmente separables como el XOR biológico.

Funciones de activación canónicas:

- **Sigmoide (σ):** $\sigma(z) = \frac{1}{1+e^{-z}}$, con codominio en $(0, 1)$. Es idónea para capas de salida en clasificación binaria porque su valor se interpreta como una probabilidad $P(y = 1 | x)$. Su derivada analítica es $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.
- **Unidad lineal rectificadora (ReLU):** $\text{ReLU}(z) = \max(0, z)$. Es el estándar en capas ocultas porque su derivada vale 1 para $z > 0$, y ahí no hay saturación. Para $z < 0$ la derivada es 0: si una unidad deja de recibir entradas positivas, su gradiente es nulo y no vuelve a aprender. A eso se le llama unidad muerta, y es la contrapartida de su ventaja.
- **Tangente hiperbólica ($\tanh$):** $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$, centrada en cero con codominio en $(-1, 1)$.

El peligro de la saturación del gradiente en la sigmoide:

Cuando el valor absoluto de la preactivación es elevado ($|z| \gg 0$), la función sigmoide se vuelve prácticamente plana ($\sigma(z) \approx 0$ o $\sigma(z) \approx 1$). En ese régimen, su derivada tiende a cero:

$$\lim_{|z| \rightarrow \infty} \sigma'(z) = \lim_{|z| \rightarrow \infty} \sigma(z)(1 - \sigma(z)) = 0$$

Esto provoca el fenómeno del **desvanecimiento del gradiente** (*vanishing gradient*), congelando el aprendizaje de los pesos en capas anteriores.

Cuando las entradas a una función sigmoide tienen gran magnitud absoluta ($|z| \gg 0$), la derivada $\sigma'(z)$ tiende a cero, provocando el desvanecimiento del gradiente y deteniendo el aprendizaje de los pesos en capas precedentes.

Se reproduce con `verification/chapter_results.sh saturation_control 10.0`.

3 Entrenamiento, pérdida y retropropagación

El entrenamiento de una red neuronal consiste en encontrar el conjunto de pesos W y sesgos b que minimicen la discrepancia entre las predicciones del modelo $\hat{y}$ y las etiquetas biológicas reales y .

3.1 Función de pérdida de Entropía Cruzada Binaria

Para tareas de clasificación binaria supervisada con $y \in \{0, 1\}$ y $\hat{y} = P(y = 1 | x) \in (0, 1)$, la función de pérdida canónica es la **Entropía Cruzada Binaria** (*Binary Cross-Entropy*, BCE), fundamentada como el negativo de la log-verosimilitud de una distribución de Bernoulli:

$$P(y | x) = \hat{y}^y (1 - \hat{y})^{1-y} \implies -\ln P(y | x) = -[y \ln(\hat{y}) + (1 - y) \ln(1 - \hat{y})]$$

$$\mathcal{L}(y, \hat{y}) = -[y \ln(\hat{y}) + (1 - y) \ln(1 - \hat{y})]$$

Esta formulación conecta directamente con la divergencia KL y la entropía cruzada formalizadas en TC-CH07. La función de pérdida de Entropía Cruzada Binaria mide la divergencia entre la etiqueta real $y \in \{0, 1\}$ y la probabilidad predicha $\hat{y}$ mediante $\mathcal{L}(y, \hat{y}) = -[y \ln(\hat{y}) + (1 - y) \ln(1 - \hat{y})]$.

3.2 Optimización por Descenso de Gradiente

El algoritmo de **Descenso de Gradiente** ajusta iterativamente cada peso en la dirección opuesta al gradiente de la función de pérdida:

$$w_{ij} \leftarrow w_{ij} - \eta \frac{\partial \mathcal{L}}{\partial w_{ij}}$$

donde $\eta > 0$ es la **tasa de aprendizaje** (*learning rate*).

El descenso de gradiente actualiza iterativamente los parámetros en la dirección opuesta al gradiente de la pérdida según $w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w}$, donde η es la tasa de aprendizaje.

Impacto de la tasa de aprendizaje η :

- Si η es excesivamente grande ($\eta = 10.0$), el algoritmo da pasos desproporcionados, sobrepasa el mínimo y diverge numéricamente.
- Si η es muy pequeña ($\eta = 10^{-4}$), cada paso es minúsculo y la convergencia se vuelve lentísima. El gradiente no es nulo: simplemente se avanza poco por iteración.

Atención. Conviene no confundir las dos cosas. Una meseta de gradiente casi nulo es un problema distinto y no se arregla cambiando la tasa de aprendizaje: procede de la saturación de las activaciones, cuando la sigmoide trabaja en sus extremos y su derivada tiende a cero, o de unidades muertas. Si el entrenamiento no avanza, la pregunta no es solo cuánto vale la tasa, sino cuánto valen los gradientes.

Una tasa de aprendizaje excesivamente grande provoca divergencia numérica u oscilaciones destructivas sobre la superficie de pérdida, mientras que una tasa excesivamente pequeña estanca la convergencia; un ajuste riguroso de hiperparámetros es indispensable para la estabilidad del entrenamiento.

Se reproduce con `verification/chapter_results.sh learning_rate_perturbation 0.1`.

3.3 El algoritmo de Retropropagación (*Backpropagation*)

El algoritmo de **Retropropagación** (Rumelhart, Hinton & Williams, 1986) calcula de forma eficiente las derivadas parciales $\frac{\partial \mathcal{L}}{\partial w_{ij}}$ aplicando la **regla de la cadena** del cálculo multivariable en sentido inverso, desde la capa de salida hacia la capa de entrada.

La retropropagación calcula las derivadas parciales exactas de la función de pérdida respecto a cada peso y sesgo aplicando la regla de la cadena del cálculo multivariable de forma recursiva desde la capa de salida hacia las capas ocultas.

4 Caso de estudio resuelto: predicción de enhancers

ESTUDIO Este caso amplía la ruta esencial al mostrar cómo traducir un problema biológico en variables, particiones y métricas auditables.

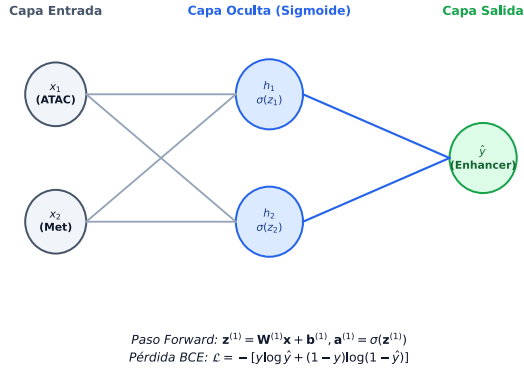
Consideremos una red neuronal feedforward con arquitectura $2 \rightarrow 2 \rightarrow 1$ diseñada para predecir si una región genómica es un **enhancer activo** ($y = 1$) a partir de dos biomarcadores epigenéticos continuos:

- $x_1 = 1.0$: Señal normalizada de enriquecimiento de la marca de histona activa H3K27ac.
- $x_2 = 0.5$: Accesibilidad cromatínica cuantificada por ATAC-seq.

Parámetros iniciales de la red:

- **Capa oculta (2 neuronas con activación sigmoide):**
 - Neurona h_1 : Pesos $w_{11}^{(1)} = 0.5$, $w_{12}^{(1)} = -0.3$; sesgo $b_1^{(1)} = 0.1$.

A) Red Neuronal Multicapa (2 -> 2 -> 1)



B) Retropropagación y Regla de la Cadena

Flujo del Gradiente (Backward Pass):

$$1. \text{ Error en Salida: } \delta^{(2)} = \frac{\partial L}{\partial \hat{y}} = (\hat{y} - y)$$

$$2. \text{ Gradiente Salida: } \frac{\partial L}{\partial \mathbf{W}^{(2)}} = \delta^{(2)} \cdot (\mathbf{a}^{(1)})^T$$

$$3. \text{ Error Oculto: } \delta^{(1)} = ((\mathbf{W}^{(2)})^T \delta^{(2)}) \odot \sigma'(\mathbf{z}^{(1)})$$

$$4. \text{ Gradiente Entrada: } \frac{\partial L}{\partial \mathbf{W}^{(1)}} = \delta^{(1)} \cdot \mathbf{x}^T$$

Actualización: $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial L}{\partial \mathbf{W}}$ | Riesgo: Saturación de $\sigma'(z)$ si $|z| \gg 0$

Figura 1: A) Arquitectura de Red Neuronal Multicapa (2 → 2 → 1) para clasificación epigenómica de enhancers activos. B) Flujo de retropropagación (*backpropagation*) y cálculo formal de gradientes mediante la regla de la cadena. Figura original adaptada de las presentaciones docentes.

- Neurona h_2 : Pesos $w_{21}^{(1)} = -0.2$, $w_{22}^{(1)} = 0.8$; sesgo $b_2^{(1)} = -0.2$.

- Capa de salida (1 neurona con activación sigmoide): Pesos $w_1^{(2)} = 0.7$, $w_2^{(2)} = -0.4$; sesgo $b^{(2)} = 0.2$.

4.1 Paso hacia adelante (*Forward Pass*)

1. Activaciones de la capa oculta:

- Preactivación de h_1 :

$$z_{h1} = w_{11}^{(1)}x_1 + w_{12}^{(1)}x_2 + b_1^{(1)} = (0.5)(1.0) + (-0.3)(0.5) + 0.1 = 0.5 - 0.15 + 0.1 = \mathbf{0.45}$$

Activación de h_1 :

$$a_{h1} = \sigma(0.45) = \frac{1}{1 + e^{-0.45}} \approx \mathbf{0.610639}$$

- Preactivación de h_2 :

$$z_{h2} = w_{21}^{(1)}x_1 + w_{22}^{(1)}x_2 + b_2^{(1)} = (-0.2)(1.0) + (0.8)(0.5) - 0.2 = -0.2 + 0.4 - 0.2 = \mathbf{0.00}$$

Activación de h_2 :

$$a_{h2} = \sigma(0.00) = \frac{1}{1 + e^0} = \mathbf{0.500000}$$

2. Activación de la capa de salida:

- Preactivación de salida:

$$z_{\text{out}} = w_1^{(2)}a_{h1} + w_2^{(2)}a_{h2} + b^{(2)} = (0.7)(0.610639) + (-0.4)(0.500000) + 0.2 \approx \mathbf{0.427447}$$

- Probabilidad predicha final ($\hat{y}$):

$$\hat{y} = \sigma(0.427447) = \frac{1}{1 + e^{-0.427447}} \approx \mathbf{0.605264}$$

En la red 2-2-1 de predicción de enhancers con entradas epigenéticas $x_1 = 1.0$ (H3K27ac) y $x_2 = 0.5$ (ATAC-seq), el paso forward calcula las preactivaciones ocultas $z_{h1} = 0.45$ ($a_{h1} = 0.610639$) y $z_{h2} = 0.00$ ($a_{h2} = 0.500000$), produciendo una preactivación de salida $z_{\text{out}} = 0.427447$ y una predicción $\hat{y} = 0.605264$.

Se reproduce con `verification/chapter_results.sh forward_enhancer_pass 1.0 0.5`.

4.2 Evaluación de la función de pérdida

Dado que la muestra observada corresponde experimentalmente a un enhancer activo ($y = 1$):

$$\mathcal{L}(1, 0.605264) = -\ln(0.605264) \approx \mathbf{0.502091}$$

Para la etiqueta real $y = 1$ (enhancer activo) y la predicción $\hat{y} = 0.605264$, la pérdida de Entropía Cruzada Binaria evalúa a $-\ln(0.605264) = 0.502091$.

Se reproduce con `verification/chapter_results.sh bce_loss 1.0 0.605264`.

4.3 Paso hacia atrás (*Backward Pass*) y actualización de pesos

Para la combinación de pérdida de entropía cruzada con salida sigmoide, el error en el nodo de salida se simplifica elegantemente a:

$$\delta_{\text{out}} = \frac{\partial \mathcal{L}}{\partial z_{\text{out}}} = \hat{y} - y = 0.605264 - 1.0 = \mathbf{-0.394736}$$

Gradientes de los pesos de la capa de salida:

- $\frac{\partial \mathcal{L}}{\partial w_1^{(2)}} = \delta_{\text{out}} \cdot a_{h1} = (-0.394736)(0.610639) \approx \mathbf{-0.241041}$
- $\frac{\partial \mathcal{L}}{\partial w_2^{(2)}} = \delta_{\text{out}} \cdot a_{h2} = (-0.394736)(0.500000) \approx \mathbf{-0.197368}$

Actualización de pesos con tasa de aprendizaje $\eta = 0.1$:

- $w_1^{(2)} \leftarrow 0.7 - (0.1)(-0.241041) = 0.7 + 0.024104 = \mathbf{0.724104}$
- $w_2^{(2)} \leftarrow -0.4 - (0.1)(-0.197368) = -0.4 + 0.019737 = \mathbf{-0.380263}$

El peso $w_1^{(2)}$ conecta la neurona oculta h_1 con la salida, y aumenta porque aumentar la contribución de h_1 sube la probabilidad predicha, que es lo que hace falta cuando la etiqueta es 1.

Atención. Ese incremento no mide la importancia de H3K27ac. La neurona h_1 mezcla las dos entradas, así que su peso hacia la salida no se puede leer como «la red ha aprendido que esta marca importa». Atribuir importancia a una variable de entrada exige métodos específicos, y aun así son estimaciones y no explicaciones. Es el error de interpretación más frecuente con redes pequeñas, y la sección sobre la operación peligrosa vuelve sobre él.

En el paso backward para $y = 1$, el error de salida δ_{out} es -0.394736 , produciendo los gradientes $\frac{\partial \mathcal{L}}{\partial w_1^{(2)}} = -0.241041$ y $\frac{\partial \mathcal{L}}{\partial w_2^{(2)}} = -0.197368$; actualizando con $\eta = 0.1$ se obtiene el nuevo peso $w_1^{(2)} = 0.724104$.

Se reproduce con `verification/chapter_results.sh backward_enhancer_pass 1.0 0.5 1.0 0.1`.

Y ahora hacia atrás otra vez: la capa oculta.

Hasta aquí solo hemos actualizado la última capa, que es la parte fácil porque su error se conoce directamente. El interés de la retropropagación está en el paso siguiente: repartir ese error entre las neuronas ocultas, cada una según cuánto contribuyó.

El error que llega a cada neurona oculta es el de la salida, multiplicado por el peso que la conecta con ella y por la derivada de su propia activación:

$$\delta_h = (W^{(2)})^T \delta_{\text{out}} \odot \sigma'(z_h)$$

Con los números del caso, $\delta_{\text{out}} = -0.394736$, $a_{h1} = 0.610639$ y $a_{h2} = 0.5$:

- $\delta_{h1} = 0.7 \cdot (-0.394736) \cdot 0.610639 \cdot 0.389361 = \mathbf{-0.065696}$
- $\delta_{h2} = -0.4 \cdot (-0.394736) \cdot 0.5 \cdot 0.5 = \mathbf{0.039474}$

Fíjese en los signos: los dos errores tienen signo contrario, porque los pesos que unen cada neurona con la salida también lo tienen. La red no ajusta las dos neuronas ocultas en la misma dirección.

Los gradientes de la primera capa son δ_h multiplicado por la entrada correspondiente, y con $\eta = 0.1$ los pesos quedan así:

- $w_{11} = 0,506570$, $w_{12} = -0,296715$
- $w_{21} = -0,203947$, $w_{22} = 0,798026$
- $b_1 = 0,106570$, $b_2 = -0,203947$

Continuando el backward hasta la capa oculta con los mismos parámetros, los errores retropropagados son $-0,065696$ y $0,039474$, y los pesos de la primera capa pasan a $0,506570$, $-0,296715$, $-0,203947$ y $0,798026$.

Se reproduce con `verification/chapter_results.sh backward_hidden 1.0 0.5 1.0 0.1`.

Comprueba. Compare la magnitud de los errores de la capa oculta con la de la capa de salida: son entre seis y diez veces menores. Multiplicar por pesos menores que uno y por derivadas de la sigmoide, que valen como mucho $0,25$, encoge el error en cada capa que se retrocede. Con dos capas es irrelevante; con veinte, es el problema del gradiente que se desvanece, y explica por qué las redes profundas necesitaron ReLU y otras ideas para poder entrenarse.

```
ALGORITMO: Red_Neuronal_Forward_Backward_2x2x1
ENTRADA:
- x = [x1, x2]: Vector de características de entrada (H3K27ac y ATAC-seq, como en el caso resuelto)
- y: Etiqueta real binaria en {0, 1}
- eta: Tasa de aprendizaje (learning rate, ej. 0.1)
- W1 (2x2), b1 (2x1), W2 (1x2), b2 (1x1): Parametros actuales de la red
SALIDA:
- y_hat: Probabilidad predicha en [0, 1]
- Perdida_BCE: Perdida de entropia cruzada binaria
- W1_nuevo, b1_nuevo, W2_nuevo, b2_nuevo: Parametros actualizados por descenso de gradiente
COMPLEJIDAD: O(1) tiempo por iteracion, O(1) espacio

1. PASO HACIA ADELANTE (FORWARD PASS):
z1 <- W1 * x + b1                # Preactivacion capa oculta (2x1)
a1 <- [sigma(z1[0]), sigma(z1[1])] # Activacion sigmoide sigma(z) = 1 / (1 + exp(-z))

z2 <- W2 * a1 + b2                # Preactivacion capa de salida (1x1)
y_hat <- sigma(z2)                # Prediccion final

Perdida_BCE <- - (y * log(y_hat) + (1 - y) * log(1 - y_hat))

2. PASO HACIA ATRAS (BACKWARD PASS):
delta_salida <- y_hat - y        # Error en salida (gradiente BCE + Sigmoide)

# Gradientes capa 2 (Salida)
grad_W2 <- delta_salida * a1^T
grad_b2 <- delta_salida

# Propagacion del error hacia la capa oculta 1
sigma_prime_z1 <- [a1[0] * (1 - a1[0]), a1[1] * (1 - a1[1])]
delta_oculto <- (W2^T * delta_salida) (producto elemento a elemento) sigma_prime_z1

# Gradientes capa 1 (Oculta)
grad_W1 <- delta_oculto * x^T
grad_b1 <- delta_oculto

3. ACTUALIZACION DE PARAMETROS (DESCENSO DE GRADIENTE):
W2_nuevo <- W2 - eta * grad_W2
b2_nuevo <- b2 - eta * grad_b2
W1_nuevo <- W1 - eta * grad_W1
b1_nuevo <- b1 - eta * grad_b1

4. RETORNO:
Retornar (y_hat, Perdida_BCE, W1_nuevo, b1_nuevo, W2_nuevo, b2_nuevo)
```

5 Sobreajuste y alta dimensionalidad en ciencias ómicas

En biomedicina computacional es ubicuo el régimen de **alta dimensionalidad** ($p \gg n$): disponemos de la expresión de $p = 20\,000$ genes o millones de variantes para una cohorte clínica de solo $n = 100$ pacientes.

En este escenario, una red neuronal profunda con miles de parámetros libres puede memorizar con facilidad el ruido muestral de entrenamiento, alcanzando un 100% de exactitud aparente pero fallando catastróficamente al predecir pacientes de un nuevo hospital (**sobreajuste** / *overfitting*).

Técnicas esenciales de regularización:

- **Regularización L_2 (Weight Decay)**: Penaliza pesos grandes sumando $\frac{1}{2}\lambda\|\mathbf{w}\|^2$ a la pérdida.
- **Desconexión aleatoria (Dropout)**: Desactiva aleatoriamente una fracción de neuronas (p. ej., 20–50%) en cada paso de entrenamiento para evitar coadaptaciones espurias.
- **Parada temprana (Early Stopping)**: Monitoriza la pérdida en un conjunto de validación independiente y detiene el entrenamiento antes de que comience el sobreajuste.

En datos ómicos, la alta dimensionalidad ($p \gg n$, miles de variables de expresión génica frente a decenas de pacientes) crea un elevado riesgo de sobreajuste, donde el modelo memoriza el ruido de entrenamiento sin generalizar a nuevas cohortes clínicas.

6 Responsabilidad, ética y límites del aprendizaje automático

Un modelo que decide sobre personas exige algo más que exactitud. Esta sección recoge las limitaciones éticas y metodológicas que hay que auditar antes de llevar un modelo de caja negra a un contexto clínico, y forma parte de los resultados de aprendizaje de la asignatura tanto como el cálculo del gradiente.

1. Correlación estadística frente a causalidad biológica:

Una red neuronal detecta asociaciones estadísticas en la distribución de entrenamiento; **no demuestra causalidad biológica ni función mecanística**. Un biomarcador con alto peso predictivo puede ser un subproducto secundario de la enfermedad o un artefacto técnico (*efecto de lote* o *batch effect* del secuenciador).

2. Sesgo poblacional y demográfico:

Si un modelo de riesgo poligénico o de respuesta a fármacos se entrena mayoritariamente sobre genomas de ascendencia europea, su rendimiento se degrada drásticamente al aplicarse sobre poblaciones históricamente subrepresentadas, pudiendo inducir disparidades diagnósticas y daños clínicos.

Las predicciones de aprendizaje automático en bioinformática reflejan correlaciones estadísticas en los datos de entrenamiento y no establecen causalidad biológica ni sustituyen la validación experimental o el juicio clínico; los modelos deben auditarse frente a sesgos demográficos y aprendizaje de atajos espurios.

3. Protocolo de validación clínica en 3 niveles y prevención de fugas:

Toda aplicación biomédica de IA debe estructurarse en tres niveles metodológicos independientes:

1. **Entrenamiento y validación interna**: Ajuste de preprocesamiento, pesos e hiperparámetros mediante validación cruzada agrupada por paciente (*GroupKFold*). En un estudio con 1000 muestras de 100 pacientes, dividir filas al azar filtra la identidad del paciente entre entrenamiento y test (*data leakage*), haciendo que la red memorice al individuo en vez de la enfermedad.
2. **Conjunto de prueba congelado (Test Set)**: Evaluación final única sobre datos retenidos desde el inicio, sin reajustar ningún hiperparámetro.
3. **Validación externa independiente**: Evaluación en una cohorte procedente de otro hospital o tecnología para comprobar la generalización real (*out-of-distribution*).

Métricas y supervisión: Reportar obligatoriamente sensibilidad, especificidad y AUC-ROC junto a intervalos de confianza, manteniendo la supervisión humana como principio innegociable.

La auditoría de riesgos de un modelo exige validación cruzada estratificada en cohortes independientes sin fugas de datos y evaluación mediante métricas clínicas completas (sensibilidad, especificidad, AUC-ROC) en lugar de la exactitud global agregada.

7 La operación peligrosa: fugas de datos y saturación

Los fallos más graves en proyectos bioinformáticos de aprendizaje automático son:

1. **Fuga de datos (*Data Leakage*):** Normalizar o seleccionar genes sobre la totalidad del dataset antes de dividir en entrenamiento y test, generando métricas optimistas falsas.
2. **Tasas de aprendizaje inadecuadas:** Divergencia silenciosa a valores NaN o estancamiento por saturación sigmoidea.

Protocolo preventivo en 3 pasos:

1. **Aislamiento del conjunto de prueba:** Congelar el test set antes de cualquier preprocesamiento, imputación o escalado.
2. **Aserción de gradientes finitos:** Comprobar tras cada época que ningún peso contenga valores infinitos o NaN.
3. **Diagnóstico de interpretabilidad:** Emplear mapas de saliencia o valores SHAP para formular hipótesis auditables sobre las variables relevantes, contrastándolas siempre con modelos de referencia (*baselines* lineales) y sin asumirlas como demostración causal o prueba de ausencia de atajos.

8 Actividad de depuración: errores en la retropropagación

Analice el siguiente fragmento, que tiene tres errores en la actualización de una red:

```
backward_step(x, y_true, y_hat, a_hidden, z_hidden, W_out, eta):
    delta_out <- y_hat - y_true

    W_out[1] <- W_out[1] - (eta * delta_out * a_hidden[1])    # Fallo 3

    delta_hidden <- W_out[1] * delta_out                      # Fallo 1
    a_hidden <- sigmoide(W_in * x + b_in)                     # Fallo 2
    devolver W_out
```

Tarea. Localice los tres y clasifíquelos: cuál produce un resultado numéricamente incorrecto desde la primera iteración, cuál lo desvía poco a poco, y cuál solo desperdicia trabajo. Compruebe cada uno contra el desarrollo del caso resuelto.

Diagnóstico.

1. **Falta la derivada de la activación.** El error que llega a una neurona oculta es el de la salida por el peso y por $\sigma'(z_h)$. Sin ese factor, el error retropropagado es varias veces mayor de lo que debe, y el signo puede ser el correcto mientras la magnitud no lo es. Es el error que da valores plausibles y equivocados.
2. **Se recalculan las activaciones con los pesos ya actualizados.** La retropropagación tiene que usar las activaciones de la pasada hacia delante que produjo esta pérdida, no unas nuevas. Recalcularlas mezcla dos estados de la red, y el efecto se acumula iteración tras iteración.
3. **Se actualiza el peso antes de usarlo para propagar.** La línea siguiente necesita W_{out} tal como estaba durante la pasada hacia delante. Al actualizarlo antes, propaga con un valor que ya no corresponde. Se arregla guardando el valor anterior, o reordenando las dos líneas.

Concepto. Los tres comparten una misma causa: la retropropagación es un cálculo sobre un estado congelado de la red. Todo lo que se actualiza durante el propio cálculo contamina el resto. Por eso las bibliotecas separan tan estrictamente el cálculo de los gradientes del paso del optimizador que los aplica.

Atención. Hay un par de cosas que a menudo se señalan como errores y no lo son. Escribir $\delta = y - \hat{y}$ en vez de $\hat{y} - y$ y después *sumar* el gradiente en lugar de restarlo da exactamente la misma actualización: los dos signos se compensan. Es confuso y conviene no escribirlo así, pero no es un fallo de cálculo. Al auditar código ajeno, compruebe el resultado antes de dar por buena una lista de defectos.

9 Síntesis operativa y tablas de diagnóstico

9.1 Tabla de diagnóstico de anomalías en entrenamiento

Síntoma observado	Causa probable	Comprobación y corrección
La pérdida devuelve NaN en las primeras épocas	Tasa de aprendizaje η excesivamente grande	Reducir η (p. ej., de 1.0 a 10^{-3})
La pérdida no desciende y los gradientes son cero	Saturación de activaciones sigmoideas ($ z \gg 0$)	Inicializar pesos con Xavier/He o usar ReLU
Casi perfecto al entrenar y azaroso en la prueba clínica	sobreajuste, con más variables que muestras	regularizar, abandonar unidades y validar fuera
El modelo predice la misma clase para todas las muestras	Desbalance extremo de clases o sesgo colapsado	Balancear pesos de clase en la pérdida o submuestrear

9.2 Tabla de síntesis con preguntas de control

Fase del proyecto	Técnica recomendada	Pregunta de control
Diseño de arquitectura	Capas densas con ReLU / salida Sigmoide	¿Se justifica la no linealidad frente a modelos lineales?
Optimización	Descenso de gradiente con BCE	¿Se verifica la estabilidad numérica de la pérdida?
Evaluación ética	Validación cruzada en cohortes externas	¿Se han descartado sesgos demográficos y fugas de datos?

10 Glosario conceptual

REFERENCIA Use este glosario para consultar componentes del aprendizaje automático y sus criterios de evaluación durante la lectura.

- **Perceptrón:** unidad de cómputo lineal con activación no lineal.
- **Sigmoide:** función $1/(1 + e^{-z})$ con codominio en $(0, 1)$.
- **ReLU:** rectificador lineal $\max(0, z)$ para redes profundas.
- **Entropía cruzada:** función de pérdida canónica para clasificación probabilística.
- **Backpropagation:** cálculo de gradientes mediante la regla de la cadena.
- **Learning rate (η):** tamaño de paso en el descenso de gradiente.
- **Vanishing gradient:** desvanecimiento de derivadas en regiones saturadas.
- **Overfitting:** memorización de ruido muestral en lugar de patrones reales.
- **Dropout:** regularización por desactivación estocástica de neuronas.
- **Data leakage:** filtración de información del test set al entrenamiento.

11 Cierre del libro: la caja de herramientas computacionales

Este capítulo final integra el conjunto de herramientas computacionales desarrollado a lo largo de todo el libro: desde la automatización reproducible en terminal Unix (TC-CH01–TC-CH02), el análisis asintótico de eficiencia y estructuras de datos (TC-CH03–TC-CH06), la cuantificación de incertidumbre por teoría de la información (TC-CH07), y los modelos probabilísticos y de alineamiento (TC-CH08–TC-CH09), hasta el aprendizaje de representaciones mediante redes neuronales.

El estudiante dispone ahora de un criterio técnico riguroso: sabe cuándo basta un script de Bash con expresiones regulares, cuándo formular un modelo probabilístico transparente con cadenas de Markov o programación

dinámica, y cuándo recurrir al aprendizaje automático supervisado, auditando siempre con responsabilidad ética y científica los límites de cada herramienta.

12 Fuentes y reproducibilidad

Este capítulo se apoya en la presentación docente de aprendizaje automático de la asignatura, escrita por Álvaro Serrano Navarro. El perceptrón sigue al artículo de Rosenblatt [2]; la retropropagación, al de Rumelhart y colaboradores [3]; y el tratamiento del sobreajuste, la regularización y las funciones de activación, al tratado de Goodfellow y colaboradores [1].

Todos los cálculos se reproducen con `verification/chapter_results.sh` tal como están impresos, con sus argumentos.

Bibliografía

- [1] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. Texto de referencia para funciones de activacion, optimizacion basada en gradientes y retropropagacion.
- [2] Frank Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958.
- [3] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.